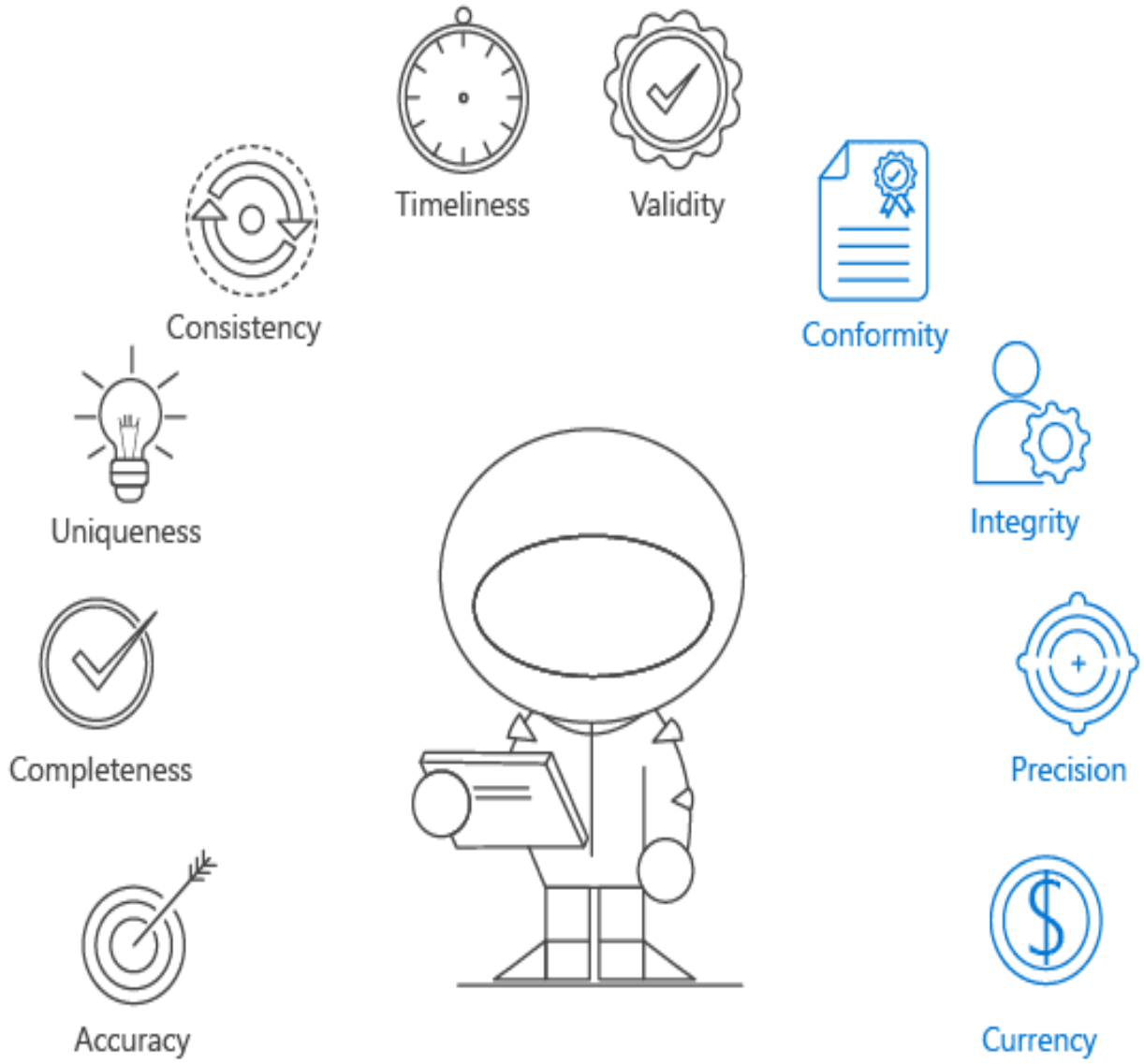
Data Preprocessing

UNIT-II

Data Quality: Why Preprocess the Data?

- Before mining, we must make the data *usable* for the specific analysis we want to do.
- In practice, real organizational data are often **incomplete, inaccurate/noisy, or inconsistent**.
- Preprocessing raises the data to the quality level your task requires.
- There are many factors comprising **data quality**, including *accuracy, completeness, consistency, timeliness,believability*, and *interpretability*.



The six key dimensions of data quality

- **Accuracy** How close the data is to the true, real-world values.
- We will take two examples to explain the data accuracy dimension and how it can be measured:
  - Data accuracy measurement with physical world
  - Data accuracy measurement with reference source

Data Accuracy Measurement with Physical World – Example

- Data accuracy can be judged by comparing data values with physical measurement or observations.

Grocery Bill		
Date		
Item	Qty	$
Sugar	1 kg	$ 5
Apple	20	$ 6



Example: We perform this data accuracy check at the grocery store every time we make a purchase – by checking the items on the bill and then physically verifying the items in the grocery cart.

CONTD…

- **Data Accuracy Measurement with Reference Source – Example**
- Another way to measure accuracy is by comparing actual values to standard or reference values provided by a reliable source.

CPI Index Value

Month	Val CPI
2020 Jan	257.971
2020 Feb	258.678
2020 Mar	258.115

Table 1. Consumer Price Index for All Urban Consumer (CPI-U): U.S. March 2020

[1982-84=100, unless otherwise noted]

Expenditure category	Relative importance Feb. 2020	Unadjusted indexes		
		Mar. 2019	Feb. 2020	Mar. 2020
All items	100,000	254,202	258,676	258,115

CONTD…

- **Completeness** – Whether all required data is present (no missing values).
  - **Example:** A salesperson wants to send an email to the customer, but the data entry operator did not fill in the email address. In this case, the data is not inaccurate, rather the email attribute was left empty. When data is missing, it directly impedes the operations of any organization.
- We will provide four examples to explain different types of data completeness quality issues:
  - The record itself is missing
  - A value in an attribute is missing
  - A reference value is missing-A system might not have all the reference values required for the domain.
  - Data truncation-the values were truncated during the loading process.

CONTD...

- **Consistency** – Data should not contradict itself across datasets/systems.
- It refers to the degree to which data values are the same across multiple datasets, systems, or repositories. It evaluates whether data remains uniform and free from contradiction regardless of where it resides within an organization's information ecosystem or external.
- Here are a few examples of Data Consistency DQ dimension:
  - Record level data consistency across source and target
  - Attribute level data consistency across source and target
  - Data consistency between subject areas
  - Data consistency in transactions
  - Data consistency across time
  - Data consistency in data representation

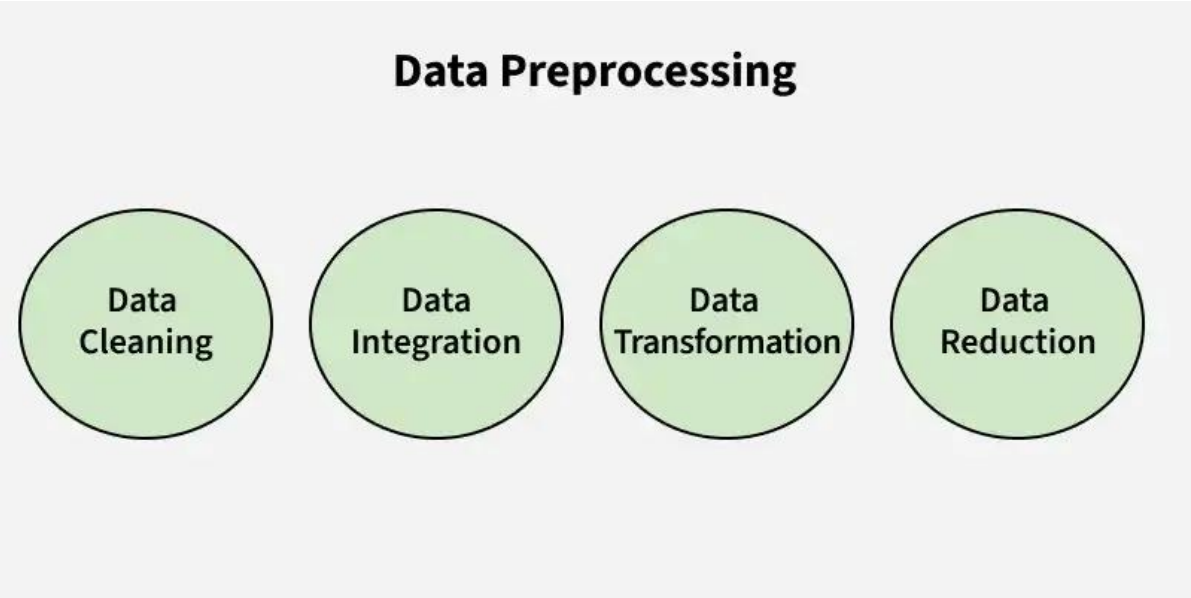
CONTD...

- **Timeliness** – Data should be up-to-date and available when needed.
  - Timeliness refers to the time lag between the actual event time and the time the event is captured in a system, making it available for use.

  Believability – users **trust** the data.
  - **Problems:** even after fixes, past errors erode confidence; people may ignore correct data they no longer believe.
- **Interpretability** – How easily data can be understood, represented, and used.
  - **Problems:** opaque codes/jargon (e.g., accounting codes) without documentation; correct data remains unusable to non-experts.

Data Preprocessing in Data Mining

- Data preprocessing is the process of preparing raw data for analysis by cleaning and transforming it into a usable format.
- In data mining it refers to preparing raw data for mining by performing tasks like cleaning, transforming, and organizing it into a format suitable for mining algorithms.
  - Goal is to improve the quality of the data.
  - Helps in handling missing values, removing duplicates, and normalizing data.
  - Ensures the accuracy and consistency of the dataset.



CONTD...

1. Data Cleaning: It is the process of identifying and correcting errors or inconsistencies in the dataset. It involves handling missing values, removing duplicates, and correcting incorrect or outlier data to ensure the dataset is accurate and reliable.

- Clean data is essential for effective analysis, as it improves the quality of results and enhances the performance of data models.
- **Missing Values:** This occur when data is absent from a dataset.
- **Noisy Data:** It refers to irrelevant or incorrect data that is difficult for machines to interpret, often caused by errors in data collection or entry.
- **Removing Duplicates:** It involves identifying and eliminating repeated data entries to ensure accuracy and consistency in the dataset.

CONTD...

2. Data Integration: It involves merging data from various sources into a single, unified dataset.

- It can be challenging due to differences in data formats, structures, and meanings.
- **Record Linkage** is the process of identifying and matching records from different datasets that refer to the same entity, even if they are represented differently.
- **Example:**
  - Dataset A → "Cust_ID = 101, Name = Ramu"
  - Dataset B → "CustomerNo = C101, FullName = A Ramu"
  - Record linkage matches them as the same person.

- **Data Fusion** involves combining data from multiple sources to create a more comprehensive and accurate dataset.

Example:

- Weather data from Source A → "Temperature = 29°C"
- Weather data from Source B → "Temperature = 29.2°C, Humidity = 80%"
- Fused record → "Temperature ≈ 29°C, Humidity = 80%" (more complete).

CONTD...

3. Data Transformation: It involves converting data into a format suitable for analysis.

- Common techniques include
- **Data Normalization**: The process of scaling data to a common range to ensure consistency across variables.
- **Example:**
  - Heights: [150, 160, 170, 180] cm
  - After Min-Max Normalization (to [0,1]): [0.0, 0.33, 0.67, 1.0]
- **Standardization (Z-score Normalization):** Adjusts data so that it has **zero mean** and **unit variance**.
- **Discretization**: Converting continuous data into discrete categories for easier analysis.
- **Example:**
  - Age (continuous): [23, 35, 47, 60]
  - Discretized: [“Young”, “Middle-aged”, “Senior”]
- **Data Aggregation**: Combining multiple data points into a summary form, such as averages or totals, to simplify analysis.
- **Example:**
  - Daily sales: [100, 120, 90, 110]
  - Aggregated weekly sales = 420

CONTD...

4. Data Reduction: It reduces the dataset's size while maintaining key information.

- This can be done through
- **feature selection**, which chooses the most relevant features,
- **Example:**
  - Dataset: {Age, Height, Weight, Income, FavoriteColor}
  - For predicting **Income**, “FavoriteColor” may be irrelevant → removed.
- **feature extraction**, which transforms the data into a lower-dimensional space while preserving important details.
- **Example:**
  - Image dataset with 1,000 pixels (features).
  - PCA reduces to 50 principal components while retaining 95% of information.
- **Dimensionality Reduction (e.g., Principal Component Analysis)**: A technique that reduces the number of variables in a dataset while retaining its essential information.
- **Numerosity Reduction**: Reducing the number of data points by methods like sampling to simplify the dataset without losing critical patterns.
- **Example:** Instead of analyzing all **1 million transactions**, take a **10% random sample** → still reveals similar patterns.
- **Data Compression**: Reducing the size of data by encoding it in a more compact form, making it easier to store and process.
- **Example:** A **10 MB image** compressed to **2 MB JPEG** with negligible quality loss.

Data Cleaning

- Real-world data tend to be incomplete, noisy, and inconsistent. *Data cleaning* (or *data cleansing*) routines attempt to fill in missing values, smooth out noise while identifying outliers, and correct inconsistencies in the data.
- **Missing Values**

1. Ignore the tuple:This method is not very effective, unless the tuple contains several attributes with missing values. It is especially poor when the percentage of missing values per attribute varies considerably. By ignoring the tuple, we do not make use of the remaining attributes' values in the tuple.

2. Fill in the missing value manually: In general, this approach is time consuming and may not be feasible given a large data set with many missing values.

3. Use a global constant to fill in the missing value: Replace all missing attribute values by the same constant such as a label like *"Unknown"* or -∞.

4. Use a measure of central tendency for the attribute (e.g., the mean or median) to fill in the missing value: For normal (symmetric) data distributions, the mean can be used, while skewed data distribution the median is used.

5. Use the most probable value to fill in the missing value: This may be determined with regression, decision tree.

Noisy Data

- **Noise** is a random error or variance in a measured variable.
- “Smooth” is the technique used on data to remove the noise. The following data smoothing techniques.
- **Binning:** Binning methods smooth a sorted data value by consulting its “neighborhood ,” that is, the values around it. The sorted values are distributed into a number of “buckets,” or *bins*.
  - In **smoothing by bin means**, each value in a bin is replaced by the mean value of the bin.
  - **smoothing by bin medians** can be employed, in which each bin value is replaced by the bin median.
  - In **smoothing by bin boundaries**, the minimum and maximum values in a given bin are identified as the *bin boundaries*. Each bin value is then replaced by the closest boundary value.

Example:

Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15
Bin 2: 21, 21, 24
Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9
Bin 2: 22, 22, 22
Bin 3: 29, 29, 29

Smoothing by bin boundaries:

Bin 1: 4, 4, 15
Bin 2: 21, 21, 24
Bin 3: 25, 25, 34

Figure 3.2 Binning methods for data smoothing.

CONTD...

- **Regression:** Data smoothing can also be done by regression, a technique which is used to fit an equation to data set.
- *Multiple linear regression* is an extension of linear regression, where more than two attributes are involved and the data are fit to a multidimensional surface.
- Linear Regression y=b+mx
  - y=predicted values
  - x=Given values
- **Example with numbers**
- Suppose we have a regression equation:
- y=50,000+3000x
- Here:
  - b=50,000 (intercept → base price)
  - m=3000 (slope → price increase per sq. ft.)
  - x = size of house (sq. ft.)
  - y = predicted house price
- If x=1000 sq. ft.:
- y=50,000+3000(1000)=3,050,000 So, the predicted price = **30.5 lakh**.
- **Outlier analysis**:Outliers may be detected by clustering, for example, where similar values are organized into groups, or “clusters.” Intuitively, values that fall outside of the set of clusters may be considered outliers

Example:

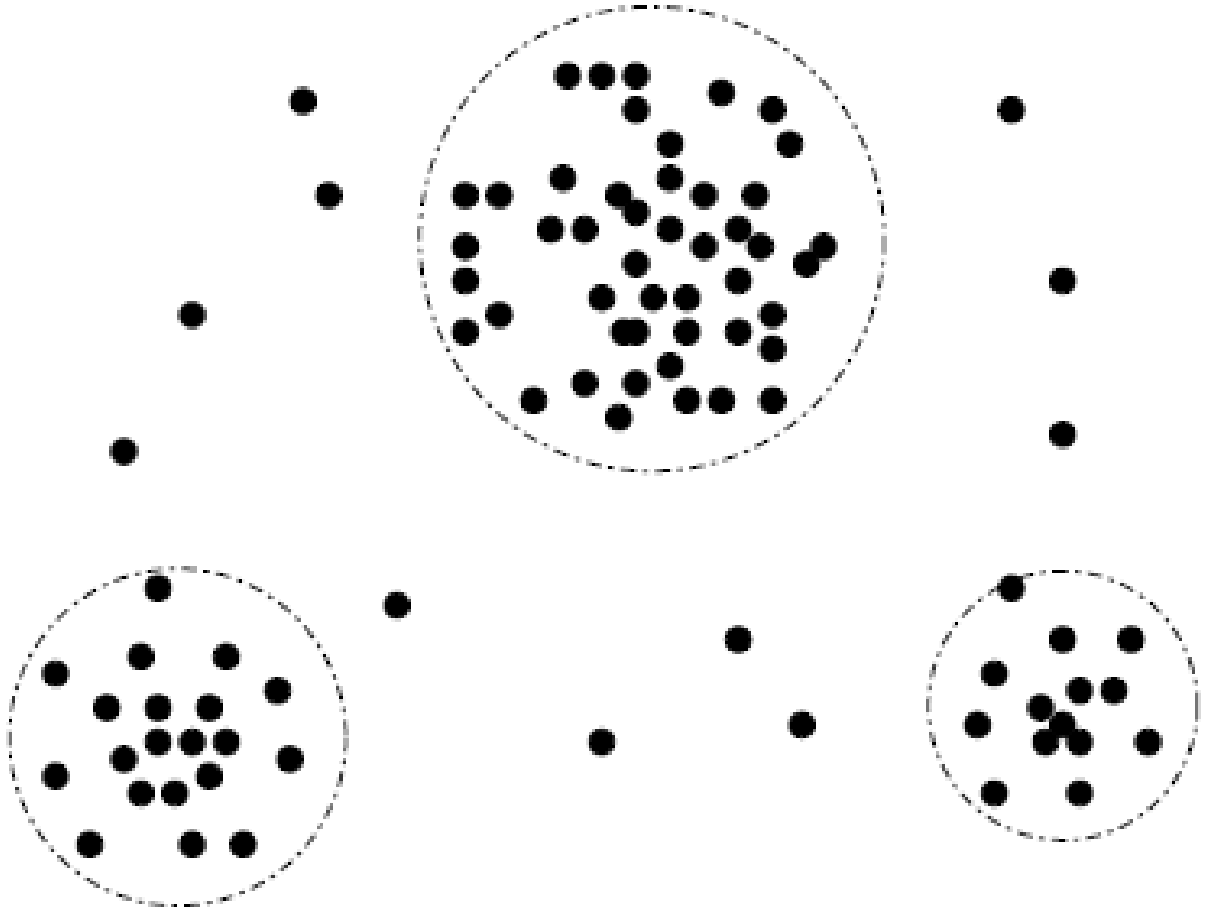


Figure 3.3 A 2-D customer data plot with respect to customer locations in a city, showing three data clusters. Outliers may be detected as values that fall outside of the cluster sets.

Data Cleaning as a Process

- Data cleaning is essential because missing values, noise, and inconsistencies make data inaccurate and unreliable for analysis.
- Steps like handling missing data and smoothing noise are important, data cleaning as a **complete process** is more comprehensive.
- It begins with **discrepancy detection**, followed by corrective actions such as transformations. This process ensures that data is consistent, accurate, and ready for analysis.

Step 1: Discrepancy Detection

- The **first step** in data cleaning is to detect discrepancies. Discrepancies may arise from many sources:
  - Poorly designed data entry forms with too many optional fields.
  - Human errors during data entry.
  - Deliberate errors, where users purposely give false or incomplete information.
  - Data decay, such as outdated contact details.
  - Inconsistent representations (e.g., “2010/12/25” vs “25/12/2010” for dates).
  - Errors from faulty devices or system failures.
  - Problems from data integration (merging data from multiple systems).

CONTD...

- Discrepancy detection uses **metadata** (data about data).
- Analysts check attributes' data types, value ranges, and domain rules to identify anomalies.
- Basic statistical methods such as mean, median, mode, range, skewness, and standard deviation help identify unusual patterns.
- **For example**, values more than two standard deviations away from the mean can be flagged as potential outliers. "two standard deviations away from the mean" means a distance of **2 × σ** above or below the mean.
- **Example:** Suppose you're analyzing employee monthly sales:
- Mean (μ) = **50 units**
- Standard deviation (σ) = **10 units**
- Range within ±2σ:
- Lower bound = 50 – 20 = **30**
- Upper bound = 50 + 20 = **70**
- So:Most employees sell between **30 and 70 units**.
- If one employee sells **90 units**, that is 4σ above mean (unusually high).
- If another sells **20 units**, that is 3σ below mean (unusually low).
  → Both would be **flagged as potential outliers**.
- Dependencies between attributes are also checked—for instance, a person's "date of joining" cannot be earlier than their "date of birth."

CONTD...

Common Error Sources in Data

- **Inconsistent codes or formats** Different systems or data sources use different formats or codes for the same attribute. This leads to confusion and errors when combining or analyzing data

Example: System A stores gender as M or F

  - System B stores gender as Male or Female
  - System C stores gender as 1 for Male and 0 for Female

- When these datasets are merged, “M” ≠ “Male” ≠ “1”, even though they mean the same. If not standardized, analysis results can be wrong.
- **Field overloading**, A field meant for one type of data is misused to store **multiple or unrelated information**

Example: Suppose there’s a field called Phone Number. Instead of only storing phone numbers:

- Some entries store phone + extension in the same field (9876543210-102).
- Others store fax numbers or alternate numbers.
- Some even store comments like "Not available" in the same field.

CONTD...

- **Rule violations**: These happen when data does not respect **predefined rules** for quality. Let's look at the three types mentioned:
  - **Unique rule**: Some fields must contain unique values—no duplicates allowed. (e.g., Employee ID).
  - **Consecutive rule**: For certain sequences, values must not have gaps or missing entries. (e.g., check numbers).
  - **Null rule**: Guidelines for representing missing values (e.g., blank, "?", "don't know").
- **Tools for Discrepancy Detection:** Several tools exist to automate or assist this process
  - **Data scrubbing tools**: Use domain knowledge (like postal address validation or spell checking) and apply parsing and fuzzy matching to clean inconsistencies.
  - **Data auditing tools**: Discover patterns, rules, and relationships in the dataset and highlight records that violate them. These tools often use **statistical analysis, correlation detection, or clustering** to identify anomalies.

CONTD...

Step 2: Data Transformation

- After discrepancies are found, transformations must be applied to correct them.
- This may include simple replacements (e.g., renaming “gender” to “sex”) or more complex transformations using **ETL (Extract, Transform, Load) tools**.
- ETL tools provide a GUI for specifying transformations but may be limited in flexibility, so analysts often write custom scripts for advanced transformations.
- The process of discrepancy detection and data transformation is **iterative**. Sometimes fixing one discrepancy reveals another.
- Because transformations can introduce new errors, multiple iterations are often necessary until the data is sufficiently clean.

CONTD...

- **Limitations of Traditional Cleaning** Traditional data cleaning processes are often:
- **Error-prone**: While fixing errors, new errors can unintentionally be introduced.

Example: Suppose we are standardizing customer names:

- Original entries: "Jon Smith", "John Smith", "J. Smith".
- Analyst decides to standardize them all to "John Smith".

- But "Jon Smith" might actually be a **different person** (not a typo). In the process of "cleaning," **a new error is introduced**
- **Time-consuming**: Traditional cleaning is done in **batch mode** (all at once). Analysts must wait for the cleaning process to complete before seeing the results.

Example:A company runs a batch process overnight to clean **10 million sales records**.

- In the morning, analysts find that the date format conversion went wrong (e.g., MM/DD/YYYY vs DD/MM/YYYY).
- The entire cleaning must be redone → wasting time and delaying analysis

- **Non-interactive**: Traditional systems log failures (like “Error in Row 10567”) but do not explain why the error happened. Analysts must investigate manually.

E**xample:** While importing sales data, the system logs:"Error: Invalid value in column ‘Price’ at row 2049."

- But it does not explain if it was because:
  - Price was negative (-50)
  - Price was missing (NULL)
  - Price had wrong format ("Free")

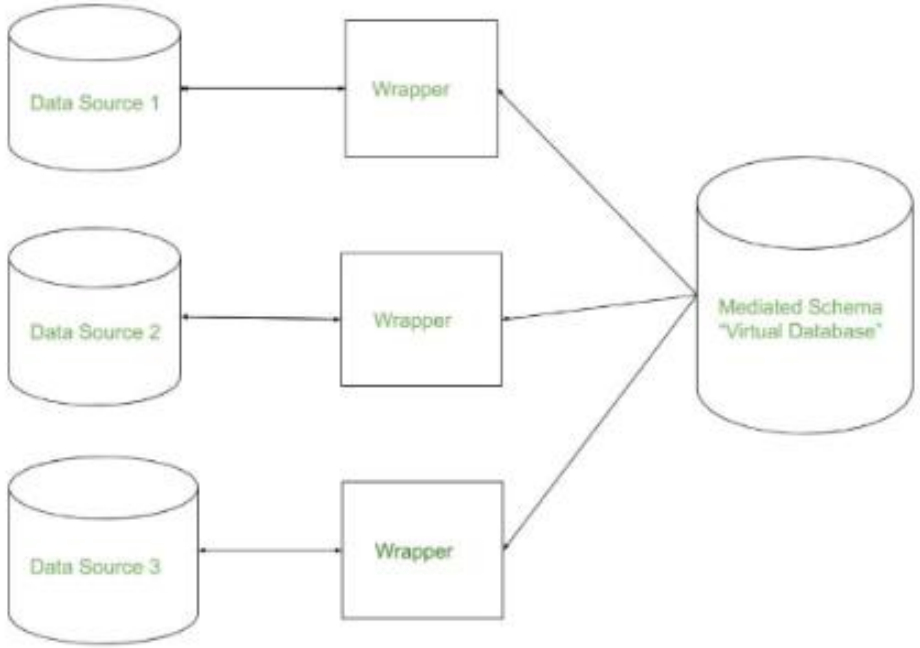
CONTD...

Modern Interactive Approaches Recent methods focus on **interactivity and feedback**. For example:

- **Potter's Wheel**: A data cleaning tool that integrates discrepancy detection and transformation in a spreadsheet-like interface. Analysts can build transformations step by step, immediately see results, undo changes, and refine transformations as they go. Discrepancy detection runs in the background automatically, helping users iteratively refine the data.
- **Declarative languages for data cleaning**: Extensions to SQL are being developed to allow analysts to specify powerful transformation rules efficiently, making the cleaning process faster and more systematic.

Data Integration

- Data integration in data mining refers to the process of combining data from multiple sources into a single, unified view.
- The goal of data integration is to make the data more useful and meaningful for the purposes of analysis and decision making.



Data Integration

- Merging of data collected from multiple sources.
- Integration helps in reducing redundancies and inconsistencies in the resulting dataset
- Approaches in Data Integration

1. Entity Identification Problem
2. Redundancy and Correlation analysis
3. Correlation Test for Nominal Data
4. Correlation Coefficient for Numeric Data
5. Covariance of Numeric Data,
6. Tuple Duplication
7. Data Value conflict Detection and Resolution

Entity Identification Problem

- In **data integration**, the **Entity Identification Problem (EIP)** arises when we need to determine whether two or more records from different data sources refer to the **same real-world entity**.
- Also known as **record linkage**, **object identification**, or **data matching**.
- **Reasons?**
  - **Different naming conventions**
  - **Spelling variations / typos**
  - **Abbreviations / short forms**
  - **Different formats** for the same attribute
  - **Missing or inconsistent values**

1) Different naming conventions

- Same entity recorded with different *name structures* or *ordering* depending on culture, system design, or product constraints.

Typical patterns & examples

- **Order differences:**
  - First Last vs Last, First
  - Sunita Rao ↔ Rao, Sunita

Company styles:

- Tata Consultancy Services ↔ TCS ↔ TCS Ltd.

2) Spelling variations / typos

- Human error or alternative spellings.
- **Insertion/Deletion/Substitution/Transposition:**
- Mohammed ↔ Muhammad (substitution)
- Srinivas ↔ Sreenivas (vowel variants)
- Kumar ↔ Kumra (transposition)
- Hyderabad ↔ Hydrabad (deletion)
- **Keyboard neighbors:** 7 vs &, I vs o

3) Abbreviations / short forms

- Shorthand, acronyms, and informal nicknames.
- Examples
- **People:** Ch ↔ Chandra, Sri ↔ Sree, V. S. Rao ↔ Venkata Subba Rao.
- **Organizations**: Intl. ↔ International, Co. ↔ Company, Pvt Ltd ↔ Private Limited.
- **Locations & addresses**: Hyd ↔ Hyderabad, Rd ↔ Road, St ↔ Street/State (ambiguous).
- **Products**: PCM 500 ↔ Paracetamol 500 mg, HDD ↔ Hard Disk Drive.

4) Different formats for the same attribute

- Same information encoded differently across systems.

Common attribute types & standardizations

- **Dates:** MM/DD/YYYY vs DD-MM-YYYY vs YYYY/MM/DD.
  - **Fix:** Parse with locale awareness; store ISO-8601 (YYYY-MM-DD).
  - **Example:** 03/07/2024 → is it 3 July or March 7? Use country context or explicit parsing rules.
- **Phone numbers:** +91-98765 43210 vs 09876543210.

5) Missing or inconsistent values

- Nulls, placeholders, partial data, or outdated/contradictory fields.
- Placeholders: N/A, -, ?, unknown, ., empty string.
- Partials: Priya without surname; Hyderabad without PIN; email missing domain.
- Inconsistencies: Two sources disagree on DOB or address; stale phone vs current phone.

Redundancy and Correlation analysis

- **Redundancy** = repeated or overlapping information at the **record** or **feature** level that doesn't add new signal and can mislead models.
- **Correlation analysis** = quantifying relationships among variables to (a) find **redundant features**,
- (b) understand **predictor–target** relationships, and
- (c) avoid issues like **multicollinearity**.

CONTD...

1) Redundancy in Data Mining

1.1 Types & where they come from

A. Instance-level redundancy (tuples/rows)

- **Exact duplicates**: identical rows after canonicalization (trim/case/whitespace).
- **Near duplicates**: same entity recorded twice with tiny differences (typos, format changes).
- **Source**: data integration, batch re-uploads, logging systems.

B. Feature-level redundancy (columns)

- **Derivable features**: Age and DateOfBirth; Price_INR and Price_USD.
- **Highly correlated features**: Height_cm vs Height_in.
- **Constant / near-constant**: zero or near-zero variance columns.
- **Leakage**: fields that directly encode or peek at the target (post-event flags).

CONTD…

- **1.2 Why redundancy is harmful**
- **Bias & double counting** (duplicate rows inflate support/confidence, skew statistics).
- **Model instability** via **multicollinearity** (coefficients swing wildly, p-values unreliable).
- **Wasted compute** and harder interpretation.
- **Overfitting** (models memorize duplicated patterns).
- **1.3 Detecting redundancy**
- **Instance-level**
- Canonicalize records → **exact dedupe** (hash keys) + **near-dedupe** (blocking + fuzzy match).

Correlation Test for Nominal Data

- **Nominal data** = categorical data with **no inherent order**.
- Examples: gender (male/female), color (red/green/blue), product category (electronics/furniture/clothing).
- So, to measure **association** between **two nominal attributes**, we use **statistical tests like Chi-Square Test of Independence**.
- **2. Chi-Square Test of Independence (χ^2 Test)**
- To check if two nominal variables are **independent** or if there is a **relationship** between them.
- Compare the **observed frequencies** (actual data) with the **expected frequencies** (if the variables were independent).

Formula

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

where

- O = Observed frequency
- E = Expected frequency

CONTD...

Gender	Bought	Didn't Buy	Total
Male	30	20	50
Female	10	40	50
Total	40	60	100

- **Null Hypothesis (H0):** Gender and Purchase are independent.
- **Expected frequency** for each cell:

Example: (Row Total × Column Total) / Grand Total

- For Male-Bought → (50 × 40) / 100 = 20

Now compute χ^2:Male-Bought: $(30–20)^2 / 20 = 5$

Male-Not Bought: $(20–30)^2 / 30 = 3.33$

Female-Bought: $(10–20)^2 / 20 = 5$

Female-Not Bought: $(40–30)^2 / 30 = 3.33$

$\chi2=16.66$

Compare with χ^2 critical value at **df = (2–1)(2–1) = 1**, at 0.05 significance → critical value = 3.84.

Since **16.66 > 3.84**, reject H0 → **Gender and Purchase are correlated**.

Observed Frequency

Gender	Bought	Didn't Buy	Total
Male	30	20	50
Female	10	40	50
Total	40	60	100

Expected Frequency

Gender	Bought	Didn't Buy
Male	(50*40)/100=20	(50*60)/100=30
Female	(50*40)/100=20	(50*60)/100=30

compute χ^2:

Gender	Bought	Didn't Buy
Male	$(30–20)^2 / 20 = 5$	$(20–30)^2 / 30 = 3.33$
Female	$(10–20)^2 / 20 = 5$	$(40–30)^2 / 30 = 3.33$

Correlation Coefficient for Numeric Data

- **Correlation** measures the **strength and direction** of the relationship between two numeric (quantitative) attributes.
- In **data mining**, correlation helps to discover patterns such as:
  - Does **income** increase with **education**?
  - Does **advertising spend** relate to **sales revenue**?

- **Formula:**

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

Where:

- x_i, y_i = values of attributes X and Y
- $\bar{x}, \bar{y}$ = mean of X and Y
- **Range of r:**
  - $+1$ → Perfect positive correlation
  - -1 → Perfect negative correlation
  - 0 → No correlation

CONTD...

Hours Studied (X)	Exam Score (Y)
2	50
4	60
6	65
8	80
10	85

Step 1: Find means

$\bar{x}=(2+4+6+8+10)/5=6$

$\bar{y}=(50+60+65+80+85)/5=68$

Step 2: Compute $(x-\bar{x})(y-\bar{y})$, $(x-\bar{x})^2$, $(y-\bar{y})^2$

X	Y	X−x̄	Y−ȳ	(X−x̄)(Y−ȳ)	(X−x̄)²	(Y−ȳ)²
2	50	-4	-18	72	16	324
4	60	-2	-8	16	4	64
6	65	0	-3	0	0	9
8	80	2	12	24	4	144
10	85	4	17	68	16	289

Step 3: Apply formula

$$r = \frac{\sum(x-\bar{x})(y-\bar{y})}{\sqrt{\sum(x-\bar{x})^2 \cdot \sum(y-\bar{y})^2}}$$

$$r = \frac{72+16+0+24+68}{\sqrt{(16+4+0+4+16)(324+64+9+144+289)}}$$

$$r = \frac{180}{\sqrt{40 \cdot 830}} = \frac{180}{\sqrt{33200}} = \frac{180}{182.17} \approx 0.99$$

Very strong **positive correlation** between **Hours Studied** and **Exam Score**.

Covariance of Numeric Data

- **Covariance** measures the **direction** of the linear relationship between two numeric attributes.

2. Formula

For two attributes X and Y with n observations:

$$\text{Cov}(X,Y) = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n}$$

Where:

- x_i, y_i = data values
- $\bar{x}, \bar{y}$ = means of X and Y

CONTD…

- **Example**
- Let’s use the same dataset as before:

Hours Studied (X)	Exam Score (Y)
2	50
4	60
6	65
8	80
10	85

Step 1: Find Means

$\bar{x}=(2+4+6+8+10)/$

$\bar{y}=(50+60+65+80+85)/5=68$

Step 2: Compute $(x-\bar{x})(y-\bar{y})$

X	Y	$X-\bar{x}$	$Y-\bar{y}$	$(X-\bar{x})(Y-\bar{y})$
2	50	-4	-18	72
4	60	-2	-8	16
6	65	0	-3	0
8	80	2	12	24
10	85	4	17	68

Step 3: Sum and Divide by n

Cov(X,Y)=72+16+0+24+685=1805=36

Cov(X,Y) = **36 (positive)** → As **Hours Studied increases, Exam Score increases**.

Tuple Duplication in Data Mining

1. What is a Tuple?

- In data mining, a **tuple** = one record (row) in a dataset.
- Each tuple consists of **attribute values** (columns).

StudentID	Name	Course	Marks
101	Ramesh	Math	85

2. What is Tuple Duplication?

•**Tuple duplication** occurs when the **same record (row) appears more than once** in a dataset.

•It may happen due to:

- Data entry errors (same record entered twice).
- Data integration from multiple sources.
- Lack of unique identifiers (Primary Key missing).

StudentID	Name	Course	Marks
101	Ramesh	Math	85
102	Sita	Science	90
101	Ramesh	Math	85
103	Arjun	English	78
102	Sita	Science	90

Here:

•Student **101, Ramesh** and **102, Sita** are repeated.

Data Value Conflict Detection and Resolution

- A **data value conflict** occurs when the **same real-world entity** has **different or inconsistent values** for the same attribute across datasets or sources.

CustomerID	Name	Age	City	Mobile
101	Ramesh	25	Hyderabad	9876543210
101	Ramesh	26	Secunderabad	9876543210

Here, **CustomerID 101** has:
•Age = 25 in one record, Age = 26 in another.
•City = Hyderabad in one record, Secunderabad in another.

3. Causes of Data Value Conflicts

1.Different Data Sources

1. Example: HR database says "Age = 25", while Medical records say "Age = 26".

2.Time Variation

1. Data may change over time (address updated, but old record still exists).

3.Human Error

1. Typo in entering value (e.g., 2500 vs 2050 salary).

4.Different Measurement Standards

1. Example: Weight recorded in **kg** vs **pounds**.

5.Incomplete Data Updates

1. One database updated, another not.

CONTD...

4. Detection of Conflicts

- **Entity Identification** → Ensure both records belong to the same real-world entity (using ID, Name, or other identifiers).
- **Data Profiling** → Find records with multiple values for the same attribute.
- **Statistical Methods** → Detect unusual/contradictory values.
- **Rule-based Checks** → Define constraints (e.g., "Age must be > 0").

5. Resolution of Conflicts: There is **no single correct solution** → depends on business rules.

- **Techniques:**
- **Rule-based Resolution**
  - Apply predefined rules.
  - Example: If conflict in "City", choose the most recent update.
- **Trusted Source Priority**
  - Give priority to a specific source (e.g., Government ID database is more reliable than user input).
- **Majority Voting**
  - If multiple sources, choose the value that appears most frequently.
- **Data Fusion / Aggregation**
  - Combine values instead of choosing one.
  - Example: Keep both Hyderabad & Secunderabad as "Twin Cities".
- **Manual Verification**
  - When automatic methods fail, send records for human review.

CONTD...

6. Example Resolution

- Conflict:
- Age = 25 (Source A) Age = 26 (Source B)
- Resolution methods:
- **Rule-based**: Take latest timestamp → Age = 26.
- **Trusted Source**: If Medical DB is more accurate, take Age = 26.
- **Majority voting**: If 3 sources → 25, 25, 26 → Final = 25.

7. Diagram (Process Flow)

Raw Data (Multiple Sources)

↓

Entity Identification (Find same entities)

↓

Conflict Detection (Inconsistent values)

↓

Conflict Resolution (Rules / Priority / Voting)

↓

Clean & Integrated Data

Data Reduction

1. Why Data Reduction?

- **Data reduction** techniques can be applied to obtain a reduced representation of the data set that is much smaller in volume, yet closely maintains the integrity of the original data.
- Large datasets (like in **AllElectronics**) → complex, time-consuming for mining.
- Goal: **Reduce data size** while **preserving key patterns & integrity**.
- Reduced dataset should give **almost the same mining results** but **faster**.

2. Major Strategies for Data Reduction

- There are **three main categories**:

A. Dimensionality Reduction : Reduces the **number of attributes (columns/features)** while keeping important information.

- **Why needed?**
  - Many attributes may be irrelevant, redundant, or noisy.
  - Example: In customer data, "Customer ID" may not be useful for prediction.
- **Techniques:**

  1. Attribute Subset Selection → Pick only relevant features.
    - Example: To predict "Loan Default", use "Income", "Credit Score", "Age" but remove "Customer Name".

  2. Principal Component Analysis (PCA) → Projects data into fewer **new dimensions** while keeping variance.

  3. Feature Transformation (Wavelets, Fourier transforms, etc.) → Convert to a new reduced form.

CONTD…

B. Numerosity Reduction: Reduce the **volume of data** by representing it in a smaller form.

- **Two categories:**
- **Parametric Methods (use models to represent data)**
  - Store only **model parameters** instead of raw data.
  - Example: Linear regression model y=mx+c stores only m and c, not all points.
  - **Log-linear models** → Used in probability distributions.
- **Non-parametric Methods (use approximation techniques)**
  - Do not assume a predefined model.
  - Examples:
    - **Histograms** → Group data into buckets.
      - Example: Age data → [0–20], [21–40], [41–60].
    - **Clustering** → Store cluster centroids instead of all points.
      - Example: Represent 10,000 customer points with 5 cluster centers.
    - **Sampling** → Analyze a representative subset.
      - Example: Use 10% of sales data for trend analysis.
    - **Data Cube Aggregation** → Store pre-computed summaries instead of raw data.
      - Example: Instead of all transactions, store monthly totals.

CONTD…

C. Data Compression: Transform the data into a compressed representation.

- **Two types:**
  - **Lossless Compression** → Original data can be perfectly reconstructed.
    - Example: String compression algorithms (Huffman coding, ZIP).
  - **Lossy Compression** → Only approximate reconstruction possible.
    - Example: JPEG image compression (loses some detail).
- **Relation to Reduction**: Dimensionality reduction and numerosity reduction are also forms of compression.

Wavelet Transforms (DWT) in Data Mining

- The **Discrete Wavelet Transform (DWT)** is a **linear signal processing technique**.
- When applied to a data vector **X = (x_1, x_2, …, x_n)**, it produces a new vector **X'** of *wavelet coefficients*.
- Importantly, **X and X' are of the same length**.
- **How Useful for Data Reduction:** After transformation, **most coefficients are small** while only a few are significantly large.
- By applying a **threshold**:
  - Keep only coefficients larger than the threshold.
  - Set others to **0**.
- The resulting vector becomes **sparse**, which:
  - Reduces storage space.
  - Speeds up computations (operations in sparse space are faster).

  Example: Suppose we transform a 1,024-length signal. After thresholding, only 50 coefficients remain significant. Instead of storing 1,024 values, we now store only ~50 values → huge reduction.

CONTD...

Noise Removal & Data Cleaning

- By discarding small coefficients (which usually represent noise), the DWT removes noise.
- Unlike smoothing, this process **does not blur the main features**.
- Makes it a strong tool for **data cleaning and denoising**.

Relation to Discrete Fourier Transform (DFT)

- Both DFT and DWT are transformation techniques.
- **DFT** uses global sines and cosines → captures frequency but not localized details.
- **DWT** uses localized wavelets → captures both **frequency** and **spatial/local detail**.
- **Compression comparison:**
    - If both keep the same number of coefficients, **DWT produces a better approximation** (higher accuracy for same compression).
    - Thus, **DWT achieves better lossy compression** than DFT.

CONTD...

Families of Wavelets

- Unlike DFT (which is unique), **many DWT families exist**.
- Examples:
  - **Haar-2 wavelet** (simple step-like wavelet).
  - **Daubechies-4, Daubechies-6 wavelets** (more complex, smoother).
- Each wavelet family has a different number of **vanishing moments**, meaning different abilities to capture trends or details.
- Shown in Figure 3.4:
- (a) Haar-2 → step function shape.
- (b) Daubechies-4 → smoother, with more coefficients.

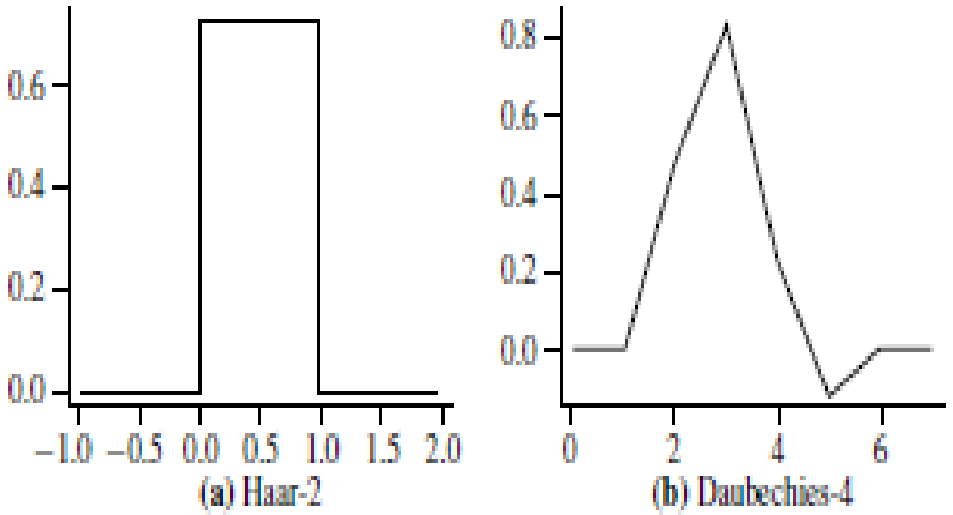


Figure 3.4 Examples of wavelet families. The number next to a wavelet name is the number of *v*
moments of the wavelet. This is a set of mathematical relationships that the coefficier
satisfy and is related to the number of coefficients.

CONTD...

General Procedure of DWT (Pyramid Algorithm)

- The DWT works in a **hierarchical, multi-resolution manner**:
- **Input vector length condition:** Must be a power of 2. If not, pad with zeros.
- **Apply two functions to pairs (x_{2i}, x_{2i+1}):**
  - **Smoothing function (low-pass):** averages → low-frequency (trend) information.
  - **Difference function (high-pass):** differences → high-frequency (detail) information.
- **Result:** Two data sets each of length L/2.
- **Repeat recursively** on the smoothed (low-frequency) part until length = 2.
- **Wavelet coefficients:** Collected from all recursive stages (low + high frequency).

Mathematical Perspective

- DWT can be expressed as a **matrix multiplication** with an **orthonormal matrix**.
- Properties:
  - Columns are orthogonal unit vectors.
  - Inverse = Transpose.
  - Enables **perfect reconstruction** of data from wavelet coefficients.
- By factorizing this matrix into sparse matrices, we get the **fast DWT algorithm**.
- **Time complexity:** O(n) → very efficient.

CONTD...

Application to Multidimensional Data

- Works not just for 1D vectors but also for **multidimensional data (like data cubes)**.
- Applied sequentially across dimensions (1st, then 2nd, etc.).
- **Complexity remains linear** w.r.t. number of cells.

Principal Components Analysis (PCA)

What is PCA?

- PCA is a method of **dimensionality reduction**.
- It finds **new axes (directions)** called **principal components** that best represent the data.
- These components are **orthogonal (perpendicular)** and fewer in number than the original attributes.
- Instead of just selecting a subset of existing attributes, PCA **creates new combined attributes**.

Why Use PCA?

- Reduces the number of dimensions → makes data easier to analyze.
- Reveals **hidden patterns or relationships** that may not be obvious.
- Helps in **visualization, clustering, regression, and noise reduction**.

CONTD...

- **PCA Procedure (Step by Step)**
- **Normalize the data**
  - All attributes are scaled to the same range → prevents large-scale attributes from dominating.
- **Find new axes (principal components)**
  - PCA calculates **k orthonormal vectors** (unit length, perpendicular to each other).
  - These form a **new coordinate system** for the data.
- **Order by importance**
  - The first principal component (**Y_1**) captures the **maximum variance** (most information).
  - The second (**Y_2**) captures the next highest variance, and so on.
  - Example in **Figure 3.5**:
    - Original data was on axes **X_1 and X_2**.
    - PCA finds new axes **Y_1 and Y_2** where Y_1 shows the most spread (variance).
- **Reduce dimensions**
  - Keep only the top principal components (those with high variance).
  - Discard weaker ones (low variance) → reduces data size but keeps most information.

CONTD...

Advantages of PCA

- Works for both **ordered and unordered data**.
- Can handle **sparse data** (lots of zeros) and **skewed data**.
- Useful for reducing **multidimensional data** to 2D for visualization.
- Often used before **clustering or regression** to improve results.

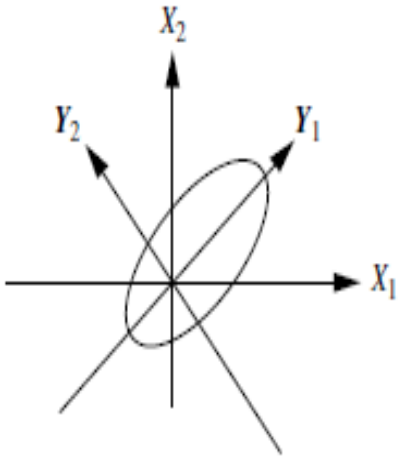


Figure 3.5 Principal components analysis. Y_1 and Y_2 are the first two principal components for the given data.

Attribute Subset Selection

- Data sets often have **many attributes** (hundreds or thousands).
- Some of them are **irrelevant** (don't affect the mining task).
  - Example: A customer's **telephone number** is irrelevant for predicting CD purchases.
- Some are **redundant** (duplicate or strongly correlated with others).
  - Example: "Date of birth" and "Age" convey the same information.
- Keeping irrelevant/redundant attributes causes:
- **Confusion** for algorithms → lower accuracy.
- **Slower mining process** due to extra dimensions.
- **Complex patterns** that are hard to understand.
- So, we want a **smaller, relevant set of attributes** that preserves the data's meaning.

Goal of Attribute Subset Selection

- Find a **minimum subset of attributes** such that the **probability distribution of classes** is as close as possible to the distribution using all attributes.
- This reduces dataset size and produces **simpler, more interpretable patterns**.

CONTD...

- **Why Not Exhaustive Search?**
- For **n attributes → 2^n possible subsets**.
- Example: If n = 20, that's **over 1 million subsets** → too expensive!
- So we use **heuristic (greedy) methods**.
  - These make the "best local choice" step by step.
  - Not guaranteed to be globally optimal, but work well in practice.
- **Greedy Heuristic Methods (Fig. 3.6)**
- **1. Stepwise Forward Selection**
- **Start**: Empty set {}.
- **Process**:
  - Add the "best" attribute one at a time (based on some evaluation measure, e.g., Information Gain, Chi-square).
- **Example from Figure**:
  - Start {} → Add A1 → Add A4 → Final reduced set {A1, A4, A6}.

2. Stepwise Backward Elimination

Start: Full set of attributes {A1, A2, A3, A4, A5, A6}.

Process:

Remove the "worst" attribute at each step.

Example from Figure:

Remove A2 → Remove A3 → Remove A5 → Final reduced set {A1, A4, A6}.

3. Hybrid (Forward + Backward)

- Combines both approaches:
  - At each step, **add the best attribute** and **remove the worst**.
- Helps balance between **overfitting (too many features)** and **underfitting (too few features)**.

4. Decision Tree Induction

Builds a **decision tree** (e.g., ID3, C4.5, CART).

At each node, the algorithm chooses the **best attribute to split data** (based on Information Gain / Gini Index).

Attributes **used in the tree = important attributes**.

Attributes **not used = irrelevant, removed**.

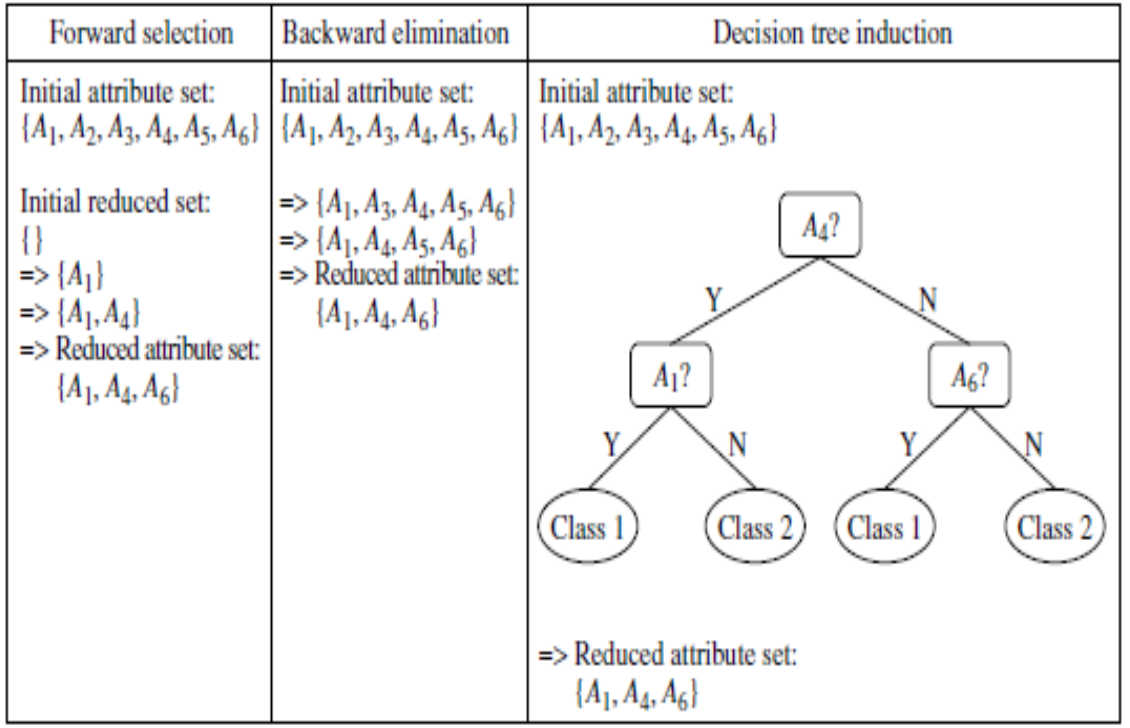


Figure 3.6 Greedy (heuristic) methods for attribute subset selection.

Example from Figure 3.6:

Root node test: A2 →

If A2 = Yes → test A1.

If A2 = No → test A6.

Reduced attribute set = {A1, A2, A6}.

Regression and Log-Linear Models: Parametric Data Reduction

- **Parametric Data Reduction**
  - Data summarized using **model parameters** (coefficients) instead of storing all raw data.
  - Provides a compact, predictive representation.
- **Regression Models**
  - Describe relationship between dependent (target) and independent (predictor) variables.
  - **Linear regression**: assumes linear relation (e.g., Y=b0+b1X1+...+bnXn).
  - Used for **continuous data reduction & prediction**.
- **Log-Linear Models**
  - Work with **categorical/multidimensional data**.
  - Model the **logarithm of expected frequencies** in contingency tables.
  - Reveal **interactions among attributes**.
- **Why Useful**
  - Compact representation (store parameters only).
  - Noise reduction (smooths random fluctuations).
  - Interpretable (equations easier than raw data).

Histograms

Definition: Histograms approximate data distribution using **binning**.

- Attribute values are partitioned into **disjoint subsets** called **buckets (bins)**.
- Each bucket stores frequency/count of values falling in its range.

Types of Buckets

- **Singleton Buckets** → Each bucket = single attribute-value pair (exact count).
- **Range Buckets** → Each bucket = continuous value range (e.g., $0–$10, $10–$20).
- **Partitioning Rules**
- **Equal-width histogram** → Buckets have same width (e.g., $10 each).
- **Equal-frequency (equal-depth) histogram** → Buckets contain roughly equal number of da

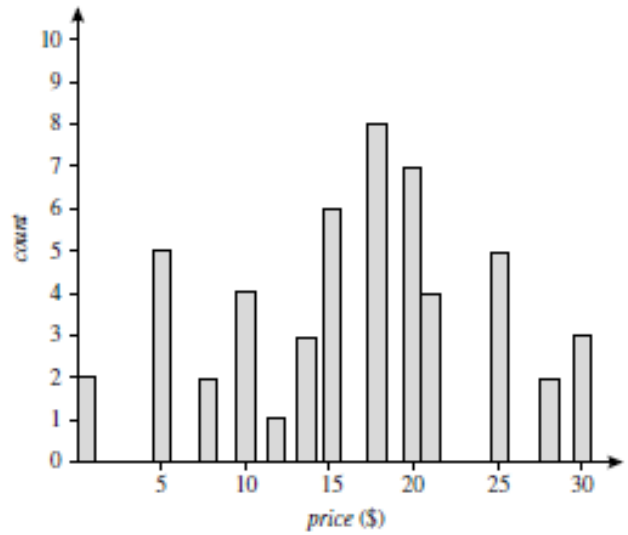


Figure 3.7 A histogram for *price* using singleton buckets—each bucket represents one price–value/frequency pair.

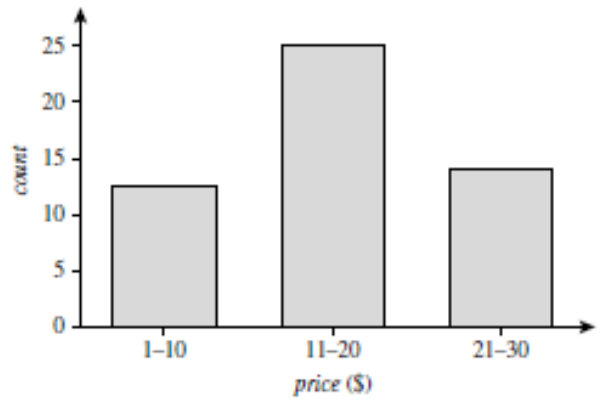


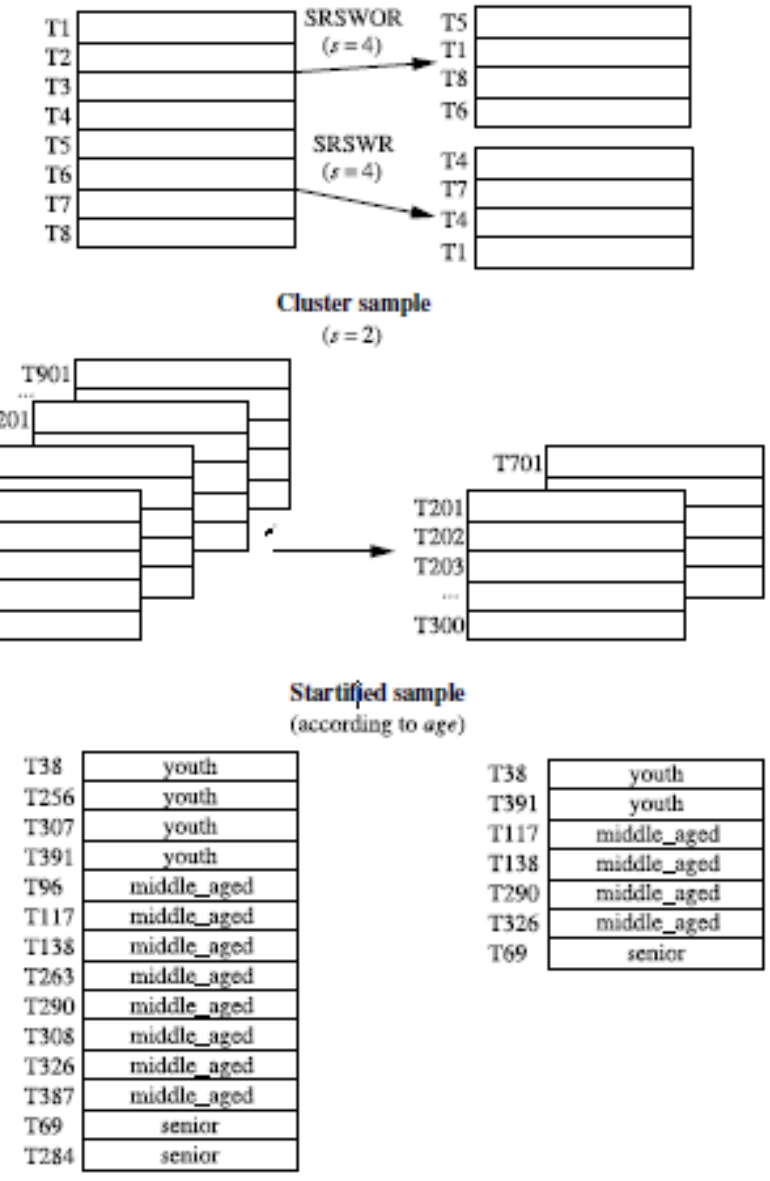
Figure 3.8 An equal-width histogram for *price*, where values are aggregated so that each bucket has a uniform width of $10.

Clustering

- **Definition:** Clustering groups data tuples (objects) into clusters where intra-cluster objects are *similar* and inter-cluster objects are *dissimilar*.
- **Similarity measure:** Usually based on a **distance function** (e.g., Euclidean, Manhattan).
- **Cluster Quality Measures:**
- **Diameter:** Maximum distance between any two objects in the cluster.
- **Centroid Distance:** Average distance of objects from the cluster centroid (the "average" object in space).

Sampling

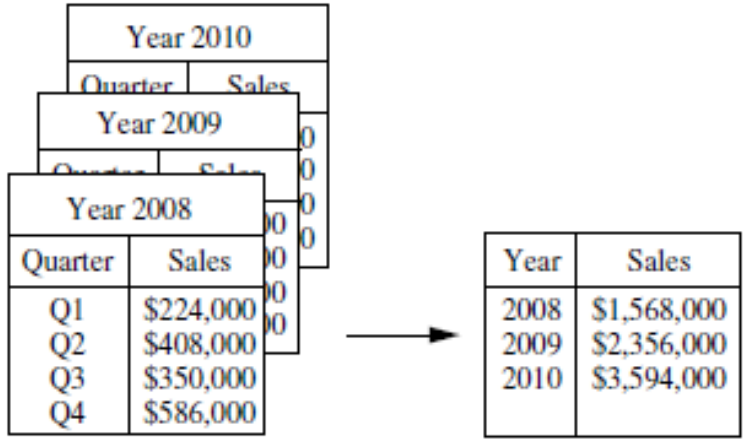
- Sampling can be used as a data reduction technique because it allows a large data set to be represented by a much smaller random data sample (or subset).
- **Simple Random Sample Without Replacement (SRSWOR):**
  - Select *s* tuples from *N* without replacement.
  - Each tuple has equal probability (1/N).
- **Simple Random Sample With Replacement (SRSWR):**
  - Similar to SRSWOR, but each selected tuple is replaced back into the dataset.
  - Same tuple can be chosen multiple times.
- **Cluster Sample:**
  - Tuples grouped into *M* disjoint clusters.
  - Randomly select *s* clusters (e.g., pages in a database or geographic clusters).
- **Stratified Sample:**
  - Data divided into disjoint strata (groups).
  - Random sample drawn from each stratum → ensures representation of all groups (especially rare ones).
- **Advantages of Sampling:**
  - Cost proportional to sample size *s* (not dataset size *N*).
  - Sampling complexity grows linearly with dimensions (*n*), while histograms grow exponentially.
  - Efficient for aggregate queries and progressive refinement (increase *s* to refine results).



e 3.9 Sampling can be used for data reduction.

Data Cube Aggregation

- A **data cube** is a multidimensional data structure that stores **aggregated information** (such as totals, counts, or averages) across different dimensions (e.g., time, location, product). Each level of the cube, called a **cuboid**, represents data at a certain level of abstraction—from the most detailed (base cuboid) to the most summarized (apex cuboid). Data cubes enable **fast, efficient analysis** of data at multiple abstraction levels, supporting OLAP and data mining tasks.
- **Aggregation for Reduction:**
  - Example: Sales data per quarter (2008–2010) → aggregated into **annual sales**.
  - Reduces data volume without losing necessary information.
- **Data Cubes:**
  - Store **multidimensional aggregated information**.
  - Each cell = aggregate value for a data point in multidimensional space (e.g., sales per item type, per branch, per year).
  - Support **fast access** to precomputed summaries → useful for OLAP and data mining.
- **Concept Hierarchies:**
  - Allow data analysis at multiple abstraction levels.
  - Example: Branches grouped into regions.
- **Cuboids:**
  - **Base cuboid:** lowest abstraction level (detailed data, e.g., per branch, per item, per year).
  - **Apex cuboid:** highest abstraction level (e.g., total sales across all branches, items, years).
  - A data cube is essentially a **lattice of cuboids**.



re 3.10 Sales data for a given branch of *AllElectronics* for the years 2008 through 2010. On the *left*, the sales are shown per quarter. On the *right*, the data are aggregated to provide the annual sales.

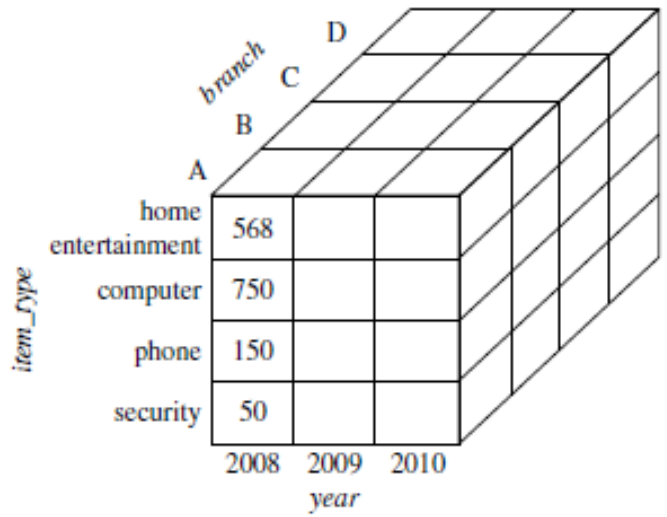


Figure 3.11 A data cube for sales at *AllElectronics*.

Data Transformation and Data Discretization

- Data Transformation Strategies

1. Smoothing Removes noise from data to improve quality and consistency.

- **Techniques**:
- *Binning*: Grouping values into intervals
- *Regression*: Fitting data to a trend line
- *Clustering*: Grouping similar data points

2. Attribute Construction (Feature Construction) Creates new attributes from existing ones to enhance mining.

- Example: attributes height and width, construct area

3. Aggregation Summarizes data to higher abstraction levels.

- **Use Case**: Often used in building data cubes for OLAP (Online Analytical Processing).
- **Example**: Daily sales → Monthly sales → Annual sales totals.

4. Normalization Scales data to a uniform range for better comparison and model convergence.

- **Common Methods**:
  - *Min-Max Scaling*: Maps values to [0, 1]
  - *Z-score Normalization*: Uses mean and standard deviation
  - *Decimal Scaling*: Moves decimal point based on value magnitude
- **Example**: Income values scaled from ₹12,000–₹98,000 to a 0–1 range.

CONTD...

5. Discretization Converts continuous numeric data into categorical intervals or labels.

- **Purpose**: Simplifies data and enables concept hierarchy generation.
- **Example**:
  - Age → [0–10], [11–20], [21–30]
  - Age → youth, adult, senior
  - Price → low, medium, high

6. Concept Hierarchy Generation for Nominal Data Generalizes nominal attributes into hierarchical levels.

- **Purpose**: Enables multi-level analysis and abstraction.
- **Example**: Street -> City -> State -> Country,Dept-> Division->Company

Data Transformation by Normalization

- **Min-max normalization**
- **z-score normalization**
- **Decimal scaling**

Discretization by Binning

- Discretization by binning is a **top-down method** for transforming continuous numerical attributes into discrete intervals (bins). This helps in reducing data complexity and preparing data for tasks like classification or data mining.
- **Binning Methods**
- **Equal-Width Binning (Interval Binning)**
  - Divide the entire range of the attribute into bins of **equal width**.
  - Formula for bin width:Width=Max value - Min value/Number of bins
  - Example: Attribute values: 10, 12, 15, 20, 25, 30
    Number of bins = 3
    - Bin 1: 10–16 → replaced by mean 12.33
    - Bin 2: 17–23 → replaced by mean 20
    - Bin 3: 24–30 → replaced by mean 27.5
- **Equal-Frequency Binning (Quantile Binning)**
  - Divide values so that **each bin contains roughly the same number of data points**.
  - Example:Attribute values: 10, 12, 15, 20, 25, 30
    Number of bins = 3
    - Bin 1: 10, 12 → replaced by mean 11
    - Bin 2: 15, 20 → replaced by mean 17.5
    - Bin 3: 25, 30 → replaced by mean 27.5